

# CELLULA FIRST WEEK

## FIRST TRY

### 1.Text Preprocessing and cleaning :

```
# Load data
df = pd.read_csv("cellula toxic data (1).csv")

texts = df["query"].astype(str)
labels = df["Toxic Category"]

le = LabelEncoder()
y = le.fit_transform(labels)

X_train, X_test, y_train, y_test = train_test_split(
    texts, y, test_size=0.2, random_state=42, stratify=y
)

# Tokenization
tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=10000, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = tf.keras.preprocessing.sequence.pad_sequences(X_train_seq, maxlen=100)
X_test_pad = tf.keras.preprocessing.sequence.pad_sequences(X_test_seq, maxlen=100)
```

1.In preprocessing we don't do cleaning or lemmatization to reduce the vocab size

### 2.The model training :

We use the lstm

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 128, input_length=100),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(len(le.classes_), activation="softmax")
])

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)

history = model.fit(
    X_train_pad, y_train,
```

## The F1 score Metrics:

```
warnings.warn(  
19/19 ————— 0s 6ms/step  
0.7052316260025491
```

## Second TRY

### 1.Text Preprocessing and cleaning :

- **Text Cleaning Function** (`preprocess_text`):
  - Convert text to lowercase.
  - Remove special characters except `.,!?` and spaces.
  - Remove extra whitespace.
  - Return empty string for non-text values.
- **Apply Cleaning:**
  - `query_cleaned` column → cleaned query.
  - `image_descriptions_cleaned` column → cleaned image descriptions.
- **Encode Target Variable:**
  - Use `LabelEncoder` to convert Toxic Category to numeric `toxic_category_encoded`.
  - Print category → number mapping.

```

import pandas as pd
import re
from sklearn.preprocessing import LabelEncoder

# 1. Load the dataset
df = pd.read_csv('cellula toxic data (1).csv')

# 2. Define a text cleaning function
def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    # Lowercase text
    text = text.lower()
    # Remove special characters but keep spaces and basic punctuation
    text = re.sub(r'^a-zA-Z0-9\s.,!?', '', text)
    # Remove extra whitespace
    text = " ".join(text.split())
    return text

# 3. Apply cleaning to text columns
df['query_cleaned'] = df['query'].apply(preprocess_text)
df['image_descriptions_cleaned'] = df['image descriptions'].apply(preprocess_text)

# 4. Encode the Toxic Category target variable
le = LabelEncoder()
df['toxic_category_encoded'] = le.fit_transform(df['Toxic Category'])

# 5. Display the label mapping
mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print("Label Mapping:", mapping)

```

✓ 0.0s

## ● Dataset Balancing:

- Count samples per class.
- Compute **average class size**.
- Resample each class to match average size (upsample smaller classes).
- Combine all classes → **df\_balanced**.
- Print new class distribution.

```

import pandas as pd
from sklearn.utils import resample

# 1 Compute the average class size (rounded to integer)
class_counts = df['toxic_category_encoded'].value_counts()
avg_count = int(class_counts.mean())
print("Average samples per class:", avg_count)

# 2 Resample each class to the average
df_balanced = pd.concat([
    resample(group,
              replace=len(group) < avg_count, # upsample small classes
              n_samples=avg_count,
              random_state=42)
    for cat, group in df.groupby('toxic_category_encoded')
])

# 3 Check the new counts
print(df_balanced['toxic_category_encoded'].value_counts())

```

✓ 0.0s

Average samples per class: 333

toxic\_category\_encoded

```

0    333
1    333
2    333
3    333
4    333
5    333
6    333
7    333
8    333

```

Name: count, dtype: int64

## ● Tokenization and Padding:

- Initialize `Tokenizer` with `num_words=10000`, `oov_token="<OOV>"`.
- Fit tokenizer on training texts (`xtrain`).
- Convert `xtrain` and `xtest` to sequences of integers.
- Pad sequences to `maxlen=100`, padding at the end (`post`).

```

# 1. Load the dataset
df = pd.read_csv('cellula_toxic_data (1).csv')

# 2. Define a text cleaning function
def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    # Lowercase text
    text = text.lower()
    # Remove special characters but keep spaces and basic punctuation
    text = re.sub(r'^a-zA-Z0-9\s.,!?', '', text)
    # Remove extra whitespace
    text = " ".join(text.split())
    return text

# 3. Apply cleaning to text columns
df['query_cleaned'] = df['query'].apply(preprocess_text)
df['image_descriptions_cleaned'] = df['image_descriptions'].apply(preprocess_text)

# 4. Encode the Toxic Category target variable
le = LabelEncoder()
df['toxic_category_encoded'] = le.fit_transform(df['Toxic Category'])

# 5. Display the label mapping
mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print("Label Mapping:", mapping)

```

## 2. Model Training:

### Model Architecture (BiLSTM):

- Embedding layer: vocabulary 10000, embedding dim 128, input length 100
- Bidirectional LSTM: 64 units, return sequences False.
- Dense layer: 64 units, ReLU activation, L2 regularization (0.01).
- Output layer: units = number of classes, softmax activation.

### Compile Model:

- Optimizer: Adam.

- Loss: `sparse_categorical_crossentropy`.
- Metric: `accuracy`.

#### **Compute Class Weights:**

- Use `compute_class_weight` to handle imbalanced classes.
- `class_weight="balanced"`
- Classes: unique values from `ytrain`.
- Map class weights to integers for training.

#### **Train Model (`model.fit`):**

- Inputs: `X_train_pad`, `ytrain`.
- Epochs: `10`
- Batch size: `32`
- Validation split: `0.1` (10%)
- Use computed `class_weight` to balance training.

```

# =====
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 128, input_length=100),
    tf.keras.layers.Bidirectional(
        tf.keras.layers.LSTM(64, return_sequences=False)
    ),
    tf.keras.layers.Dense(64, activation="relu", kernel_regularizer=tf.keras.regularizers.l2(0.01)),
    tf.keras.layers.Dense(len(le.classes_), activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# =====
# Training
# =====
from sklearn.utils.class_weight import compute_class_weight

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(ytrain),
    y=ytrain
)

history=model.fit(
    X_train_pad,
    ytrain,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    class_weight=dict(enumerate(class_weights))
)

```

✓ 10.9s

The f1score metric:

```
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.legend()
plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.show()
```

✓ 0.85

19/19 0s 10ms/step  
Weighted F1-score: 0.8536264487568832

